The Machine Learning Landscape

Hands-On Machine Learning (Géron) - Chapter 1

Giacomo Fiumara

Introduction

Machine Learning in Everyday Life

- Machine learning is used daily in applications such as spam filtering, OCR, voice assistants, translation, and recommendations.
- Early mainstream ML success: spam filters that learned to detect unwanted emails with minimal user intervention.
- ML now quietly powers many features: voice prompts, image search, product suggestions, and more.

What This Chapter Covers

- Clarifies what machine learning is and why it is useful.
- Introduces main regions of the "ML map": supervised vs unsupervised, online vs batch, instance-based vs model-based learning.
- Presents a typical ML project workflow and main challenges in practice.
- Explains how to evaluate, validate, and fine-tune ML systems.

What Is Machine Learning?

Defining Machine Learning

- Machine learning is the science and art of programming computers so they can learn from data.
- Classic definition (Arthur Samuel, 1959): field of study that gives computers the ability to learn without being explicitly programmed.
- Engineering-oriented definition (Tom Mitchell, 1997): a program learns from experience E on task T with performance measure P if its performance on T, measured by P, improves with E.

The Spam Filter Example

- Task T: classify new emails as spam or ham (non-spam).
- Experience E: a training set of labeled emails (examples tagged as spam or ham).
- Performance measure P: accuracy, i.e., proportion of correctly classified emails.
- The core learning component is a model that learns patterns from the training set and predicts labels for new emails.

Basic Terminology

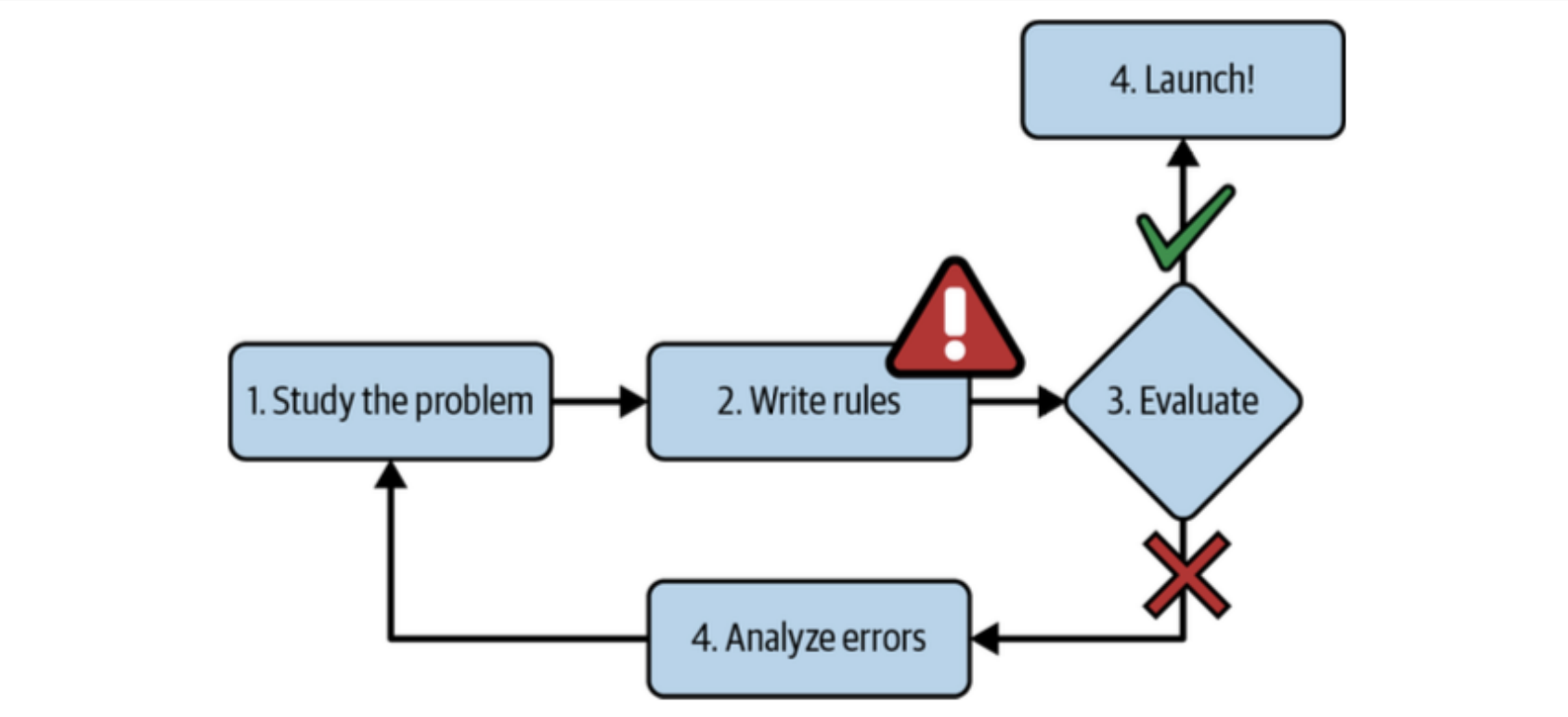
- Training set: collection of examples the system uses to learn.
- Training instance (sample): a single example from the training set.
- Model: part of the system that captures patterns in data and makes predictions (e.g., neural networks, random forests).
- Simply downloading more data (e.g., all of Wikipedia) is not learning unless it improves performance on a specific task.

Why Use Machine Learning?

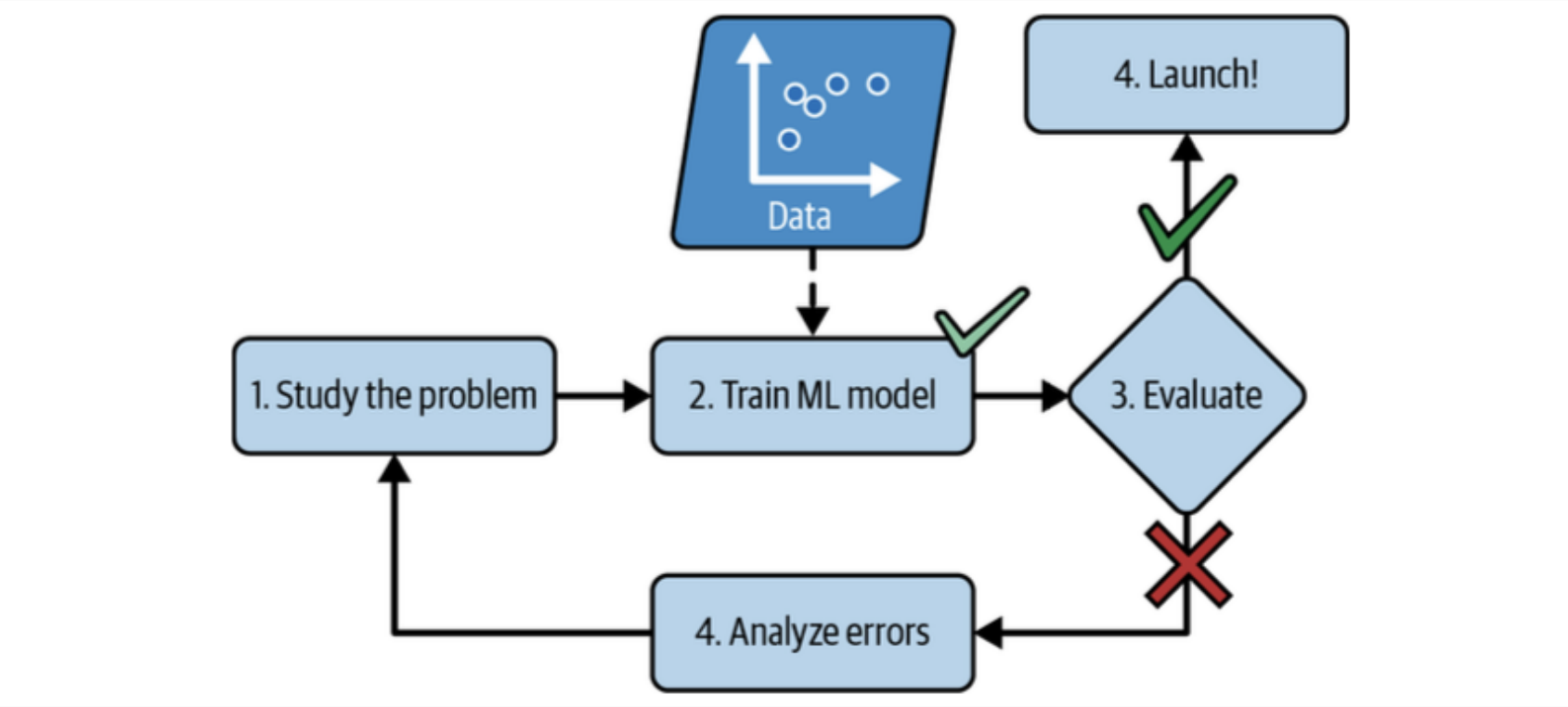
Traditional Programming vs ML – Idea

- Traditional approach: engineers handcraft rules to map inputs to outputs (e.g., many if–else rules to detect spam).
- Machine learning approach: system automatically discovers patterns in data that predict the desired output.
- For many real problems, explicit rules are hard to write, brittle, or impossible to maintain.

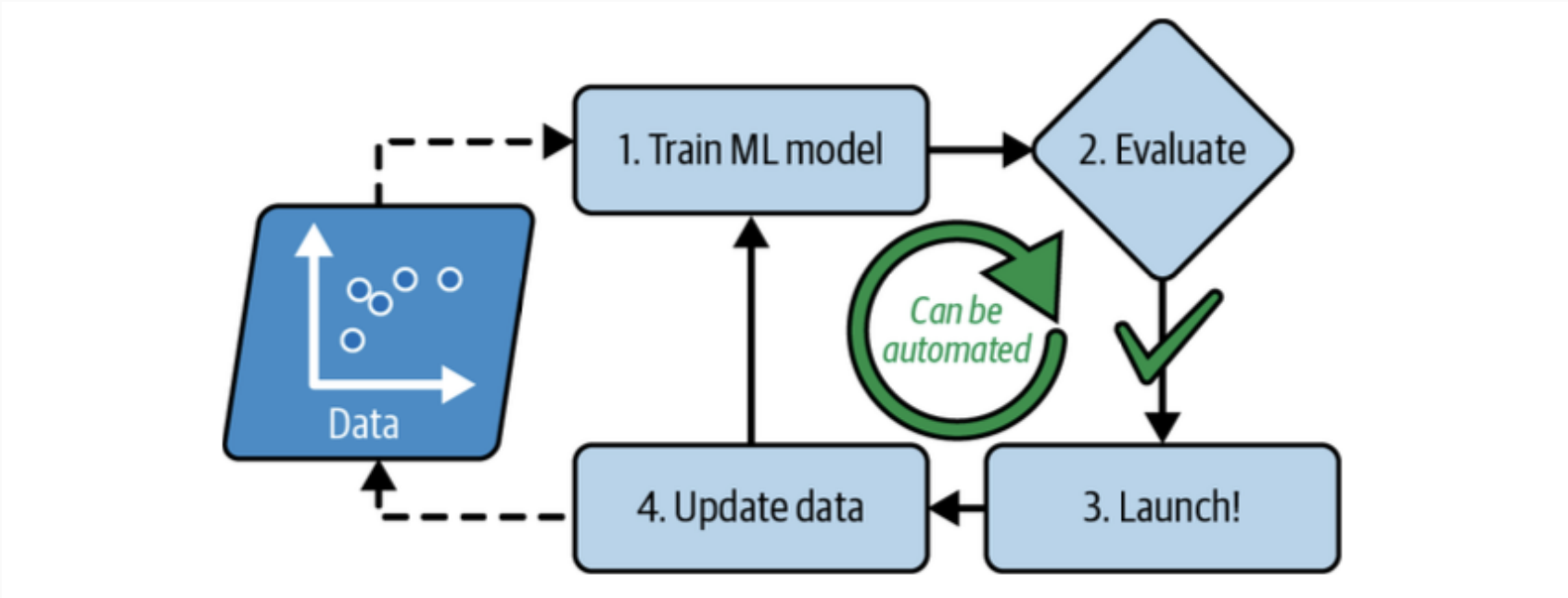
Traditional approach – manually designed rules to detect spam



Machine learning approach – model learns



ML-based spam filter adapts to new patterns



ML vs Handcrafted Rules

- A rule-based spam filter becomes a long, complex list of conditions that is hard to maintain and adapt.
- An ML-based filter learns directly from examples, often yielding shorter, more accurate, and easier-to-maintain code.
- When spammers change tactics, an ML model can adapt automatically by retraining on updated labeled examples.

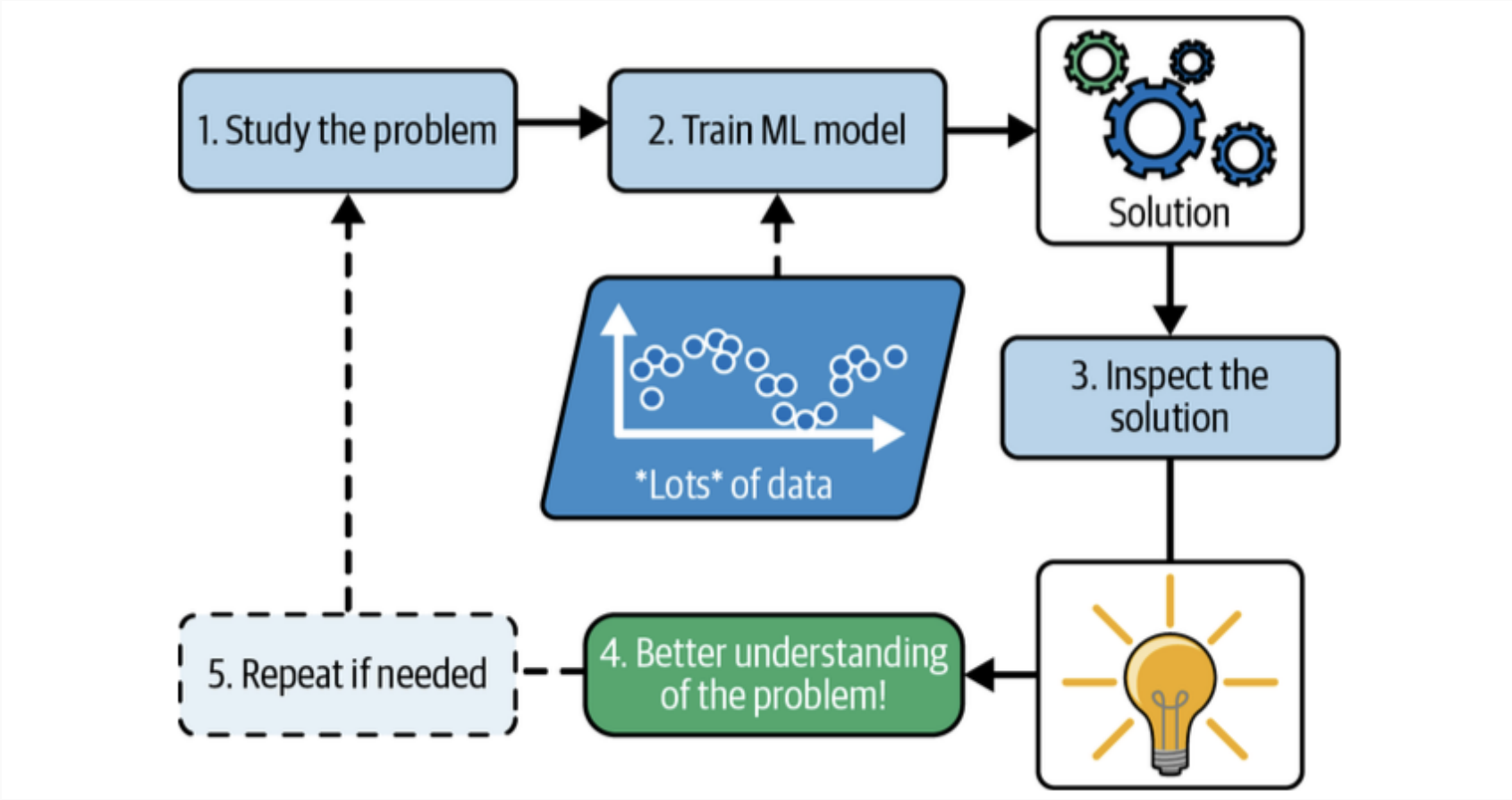
Complex Problems Without Known Algorithms

- Some tasks (e.g., large vocabulary speech recognition, robust image recognition) have no simple, explicit algorithmic solution.
- Heuristic, hand-coded approaches may work for tiny cases but fail to scale to many words, speakers, or conditions.
- ML learns from many labeled examples (e.g., audio recordings with transcripts) to handle variability and noise.

ML Helping Humans Learn

- Once trained, models can be inspected to reveal which features are most predictive (e.g., words that best indicate spam).
- Such inspection can reveal unexpected correlations or trends and improve human understanding of the domain.
- Discovering patterns in large datasets is called data mining, and ML is a key tool for it.

Machine learning surfaces informative patterns in data



When Machine Learning Shines

- Problems where existing solutions rely on many fine-tuned rules can often be simplified and improved with ML.
- Problems too complex for hand-designed algorithms may be solvable using state-of-the-art ML techniques.
- Scenarios with changing environments benefit from retrainable models that keep performance up to date.
- ML is powerful for extracting insights and patterns from large, complex datasets.

Examples of Applications

Sample ML Tasks and Techniques

- Image classification: assign labels to product images on a production line, often using convolutional neural networks (CNNs) or transformers.
- Tumor detection in brain scans: semantic segmentation classifying each pixel, typically with CNNs or transformers.
- Text classification: categorize news articles or comments, using RNNs, CNNs, or transformers.
- Text summarization: compress long documents into concise summaries with specialized NLP models.

More Application Examples

- Chatbots and assistants: integrate natural language understanding and question-answering components.
- Revenue forecasting: regression using models such as linear regression, SVMs, random forests, or neural networks.
- Voice-command apps: speech recognition using models that handle long audio sequences (RNNs, CNNs, or transformers).
- Fraud detection: anomaly detection with methods like isolation forests, Gaussian mixtures, or autoencoders.

Clustering, Visualization, Recommenders, RL

- Customer segmentation: clustering clients by purchase patterns using algorithms like k-means or DBSCAN.
- Data visualization: map high-dimensional data to 2D/3D plots using dimensionality reduction techniques.
- Recommender systems: predict next likely purchases from past sequences, often via neural networks.
- Game-playing agents: reinforcement learning trains agents (e.g., AlphaGo) to maximize long-term rewards.

Types of Machine Learning Systems

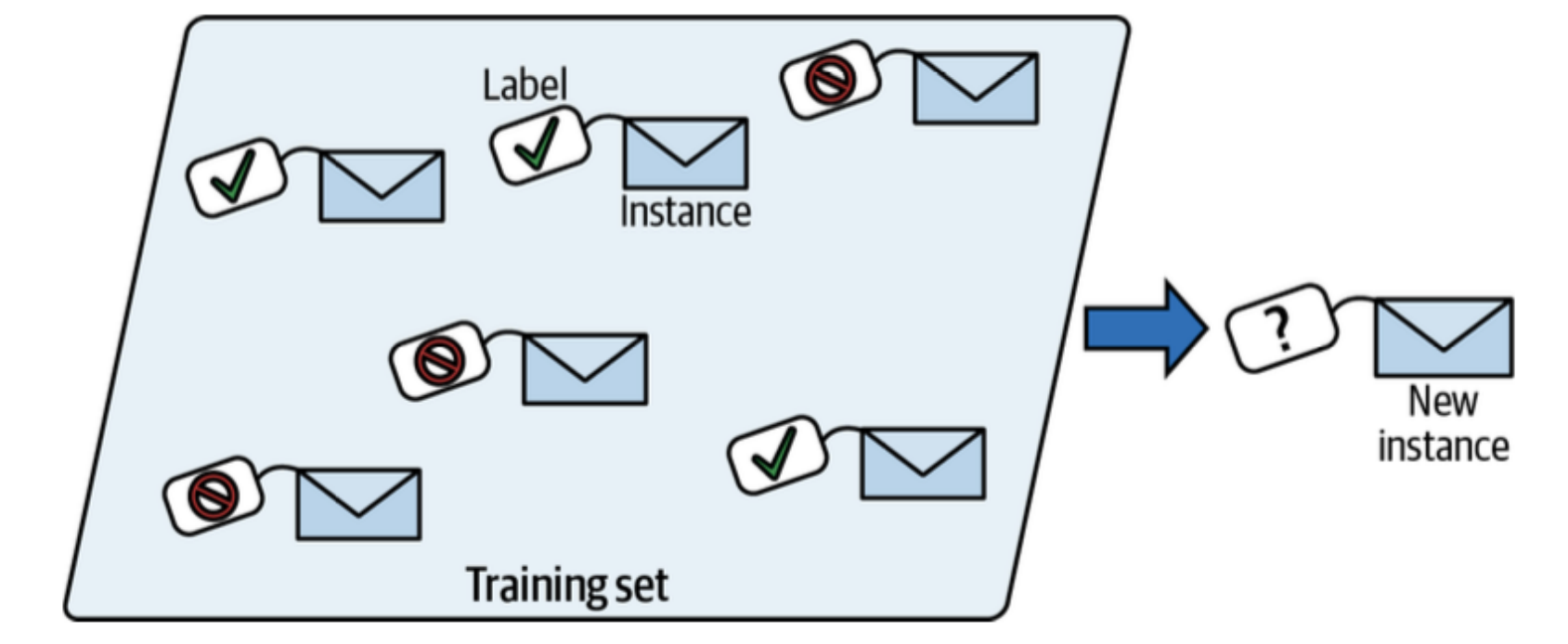
Key Classification Criteria

- Training supervision: supervised, unsupervised, semi-supervised, self-supervised, reinforcement learning.
- Learning mode over time: batch learning vs online learning.
- Generalization mechanism: instance-based vs model-based learning.
- These dimensions can be combined: a system can be, for example, online, model-based, and supervised.

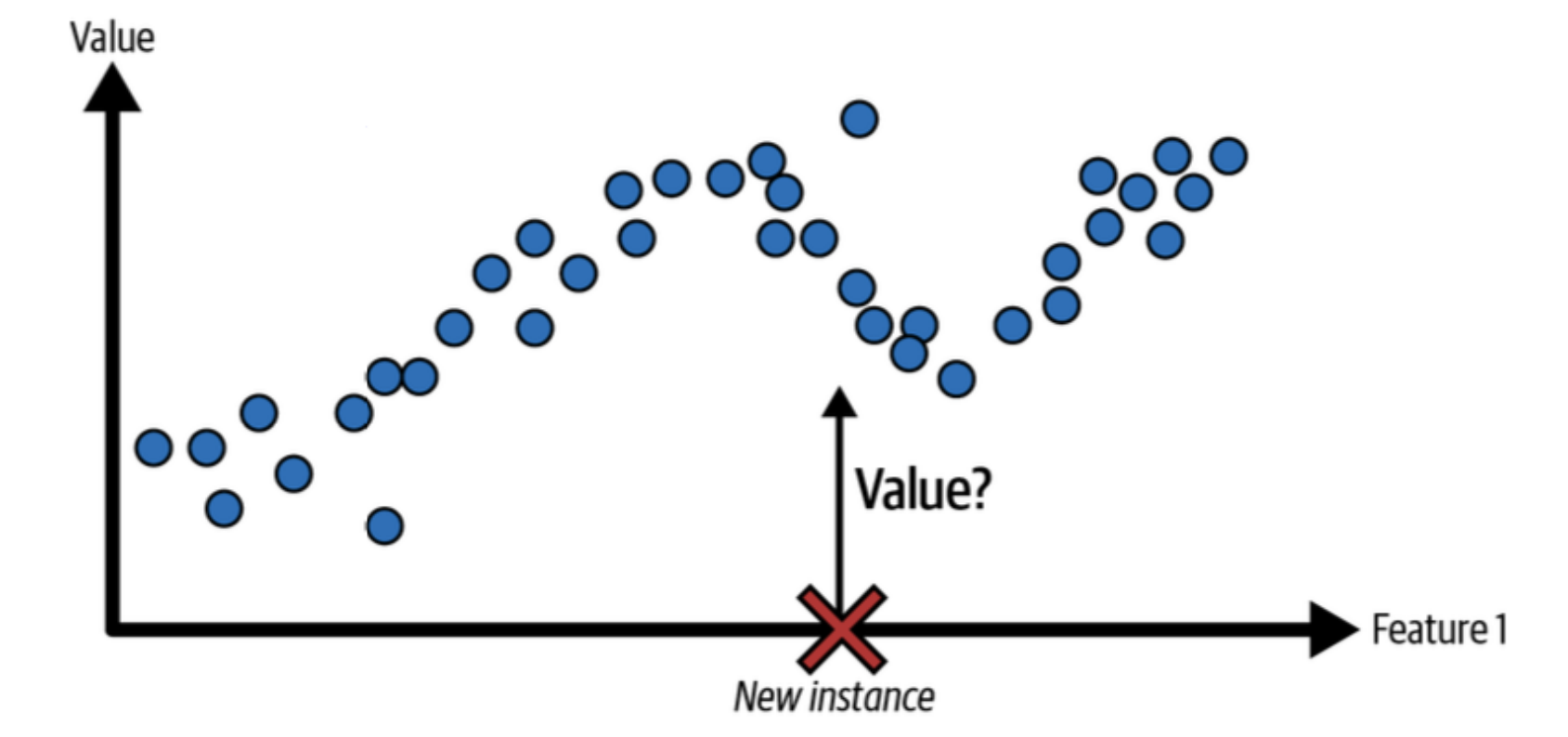
Supervised Learning

- Training data includes inputs and desired outputs, called labels, for each example.
- The system learns to map inputs to outputs so it can predict labels for new instances.
- Typical supervised tasks:
- Classification: predict discrete labels (e.g., spam vs ham).
- Regression: predict continuous values (e.g., house prices).

Labeled spam classification training set



Regression problem – predict a numeric output



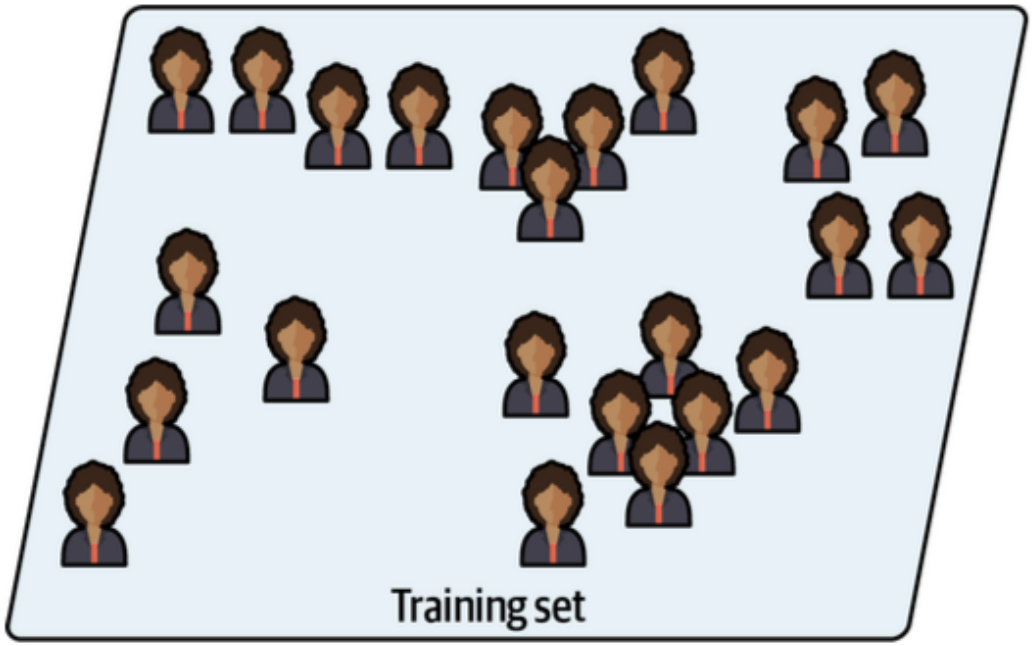
Targets, Labels, and Features

- Target and label both name the desired output; “target” is common in regression, “label” in classification.
- Features (also called predictors or attributes) are the input variables used by the model.
- Features and labels can be discussed for one instance (e.g., this car’s mileage) or for the whole dataset.

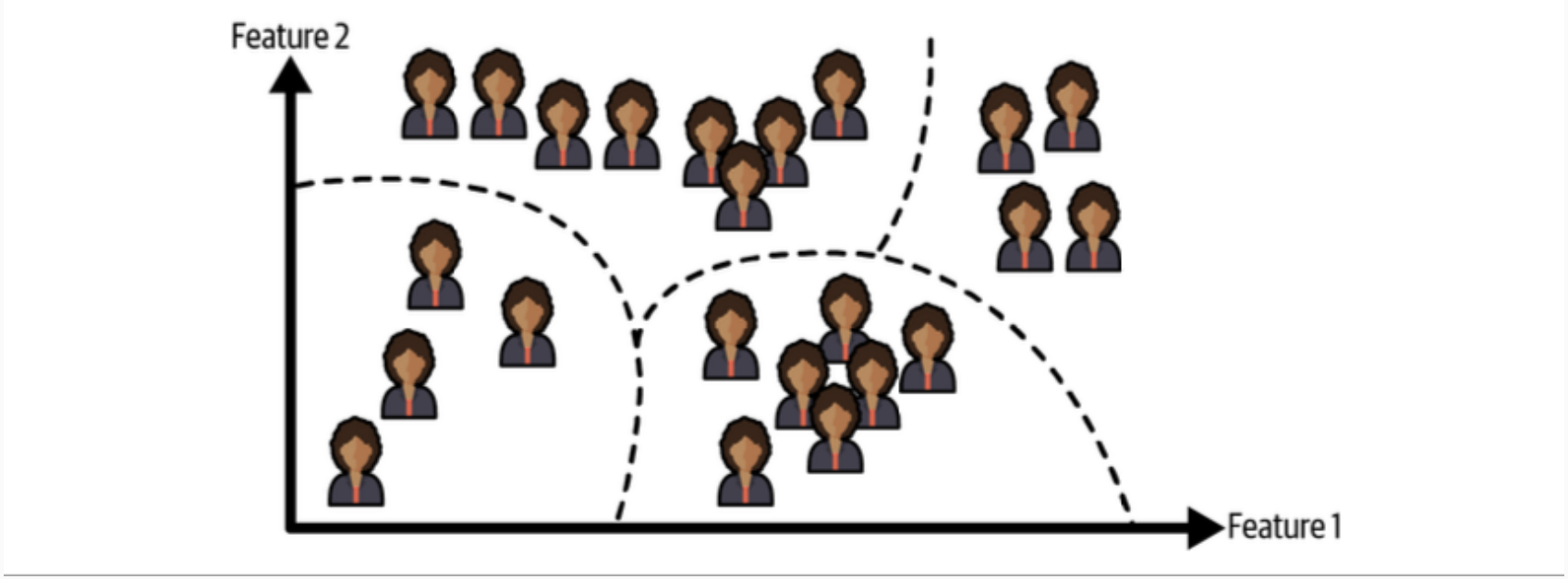
Unsupervised Learning

- Training data is unlabeled: only inputs are given, no explicit targets.
- The system discovers structure in the data on its own, such as groups or lower-dimensional representations.
- Common unsupervised tasks: clustering, visualization, dimensionality reduction, anomaly detection, association rule learning.

Unlabeled training set



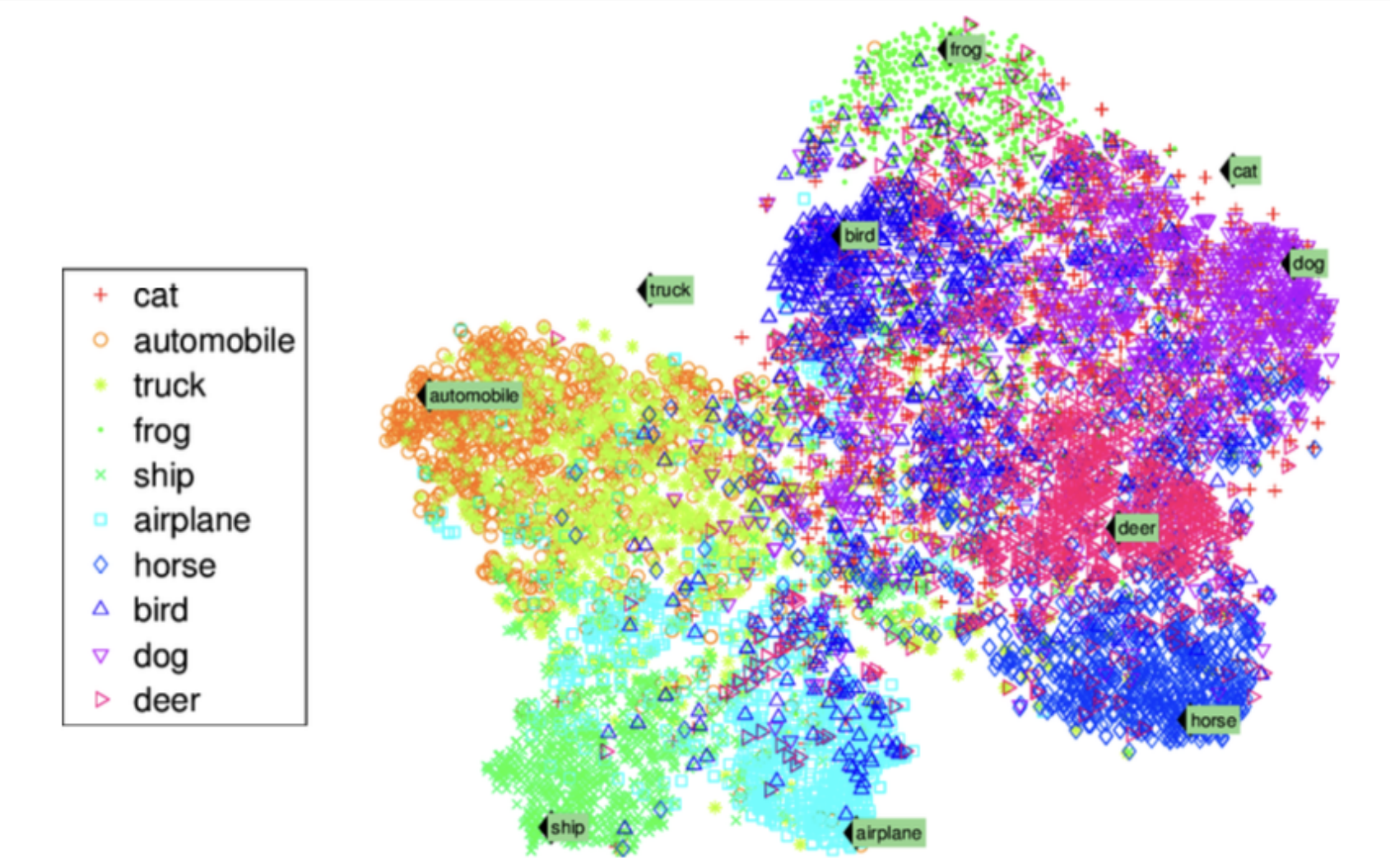
Clustering



Visualization and Dimensionality Reduction

- Visualization algorithms map high-dimensional data to 2D or 3D while preserving structure, enabling plots.
- Dimensionality reduction simplifies data by reducing the number of features while retaining important information.
- Feature extraction merges correlated features into single, more informative features (e.g., mileage and age into "wear").

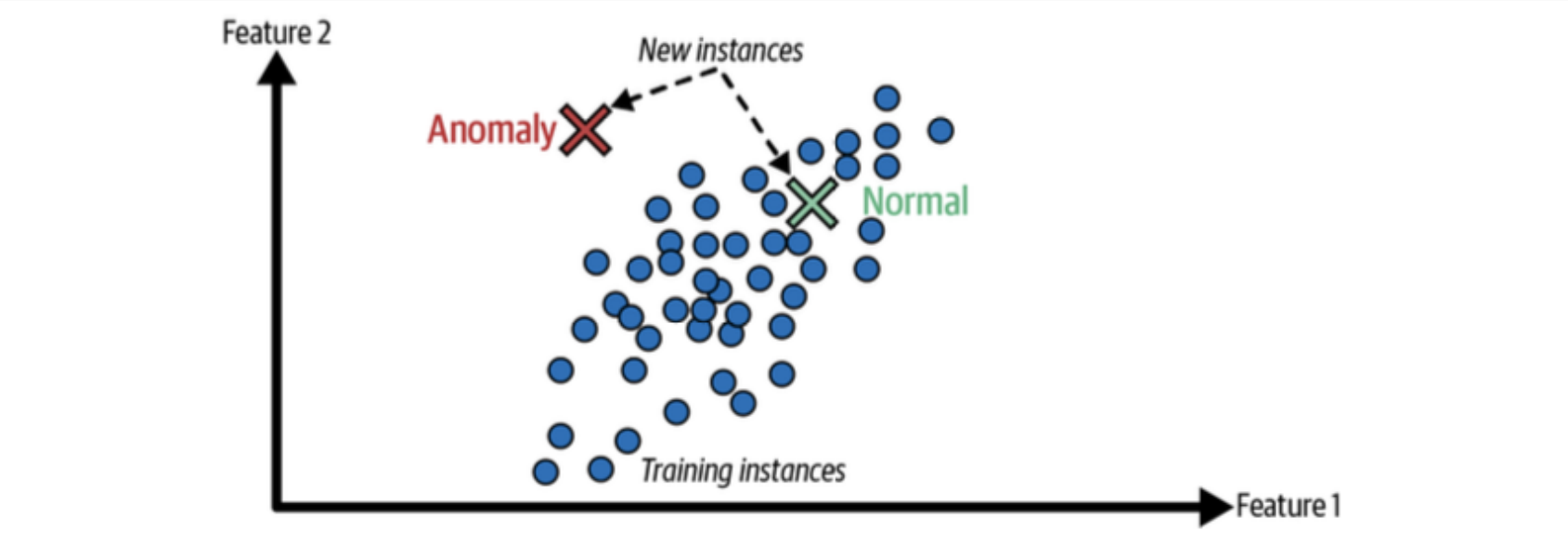
t-SNE visualization



Anomaly and Novelty Detection

- Anomaly detection: identify instances that deviate strongly from most of the training data (e.g., fraud, defects).
- Novelty detection: detect genuinely new types of instances, assuming a training set free of those novelties.
- Example: a model trained on common dog images may treat rare dog types as anomalies unless trained to see them as normal.

Anomaly detection



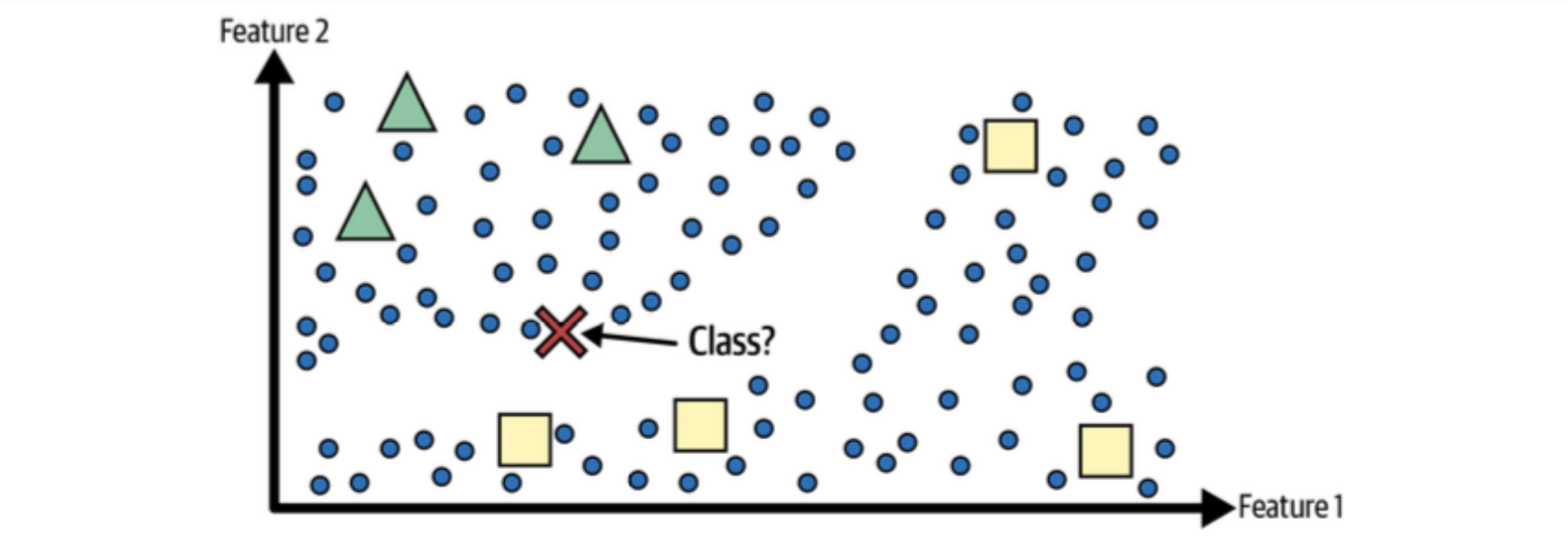
Association Rule Learning

- Goal: find interesting relations between attributes in large datasets (e.g., in market basket data).
- Example: customers buying barbecue sauce and chips often also buy steak, suggesting product placement strategies.
- These rules are learned without labels; the algorithm searches for frequent co-occurrences.

Semi-Supervised Learning

- Real datasets often have many unlabeled instances and relatively few labeled ones due to labeling cost.
- Semi-supervised learning uses both unlabeled and labeled data together to improve learning.
- Typical approach: cluster unlabeled data, then assign cluster labels based on a few labeled examples per cluster.

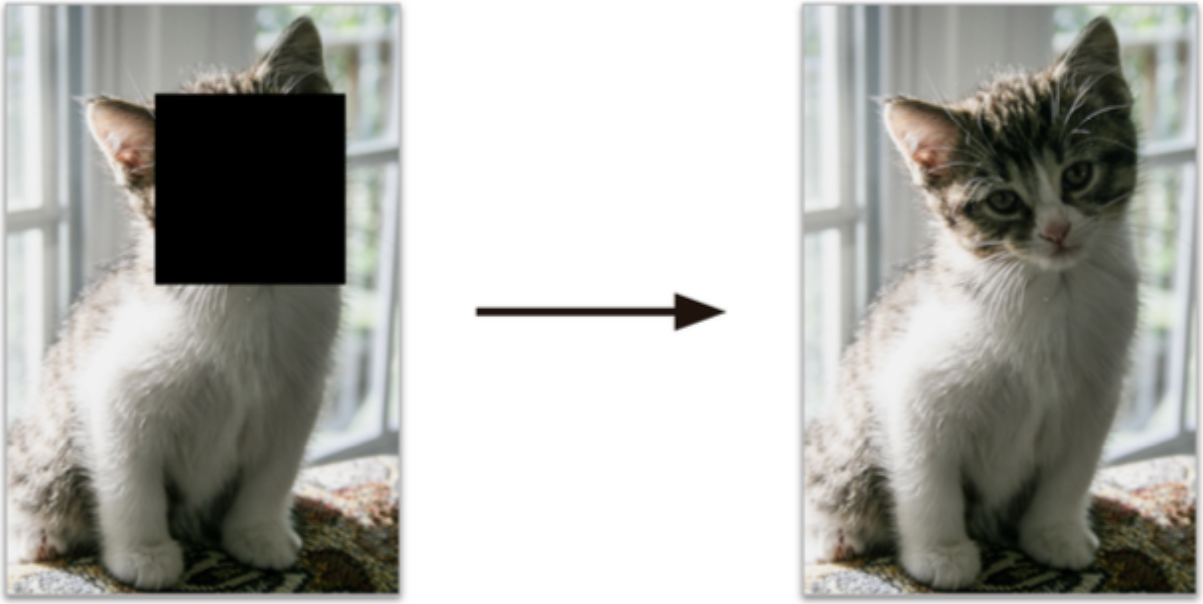
Semi-supervised learning



Self-Supervised Learning

- Self-supervised learning generates labeled data automatically from purely unlabeled data.
- Example: mask parts of images and train a model to reconstruct the original image, using corrupted images as inputs and originals as targets.
- The resulting model often serves as a starting point (via transfer learning) for downstream supervised tasks.

Self-supervised training



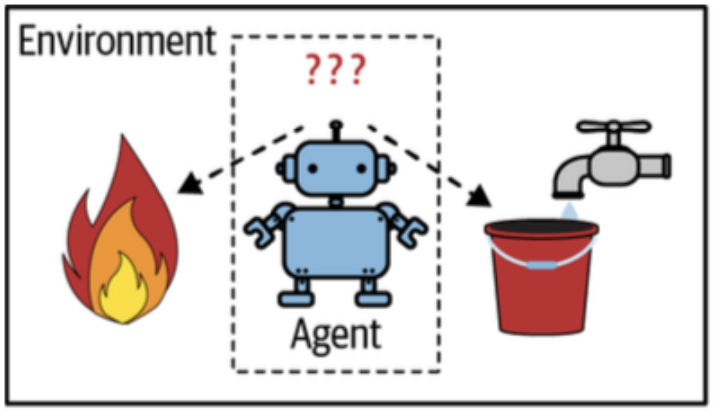
Self-Supervision and Transfer Learning

- Self-supervised models learn useful internal representations (e.g., shapes and textures of pets).
- These representations can be adapted to new tasks (e.g., pet species classification) with relatively small labeled datasets.
- Transfer learning reuses knowledge from one task to accelerate and improve learning on a related task.

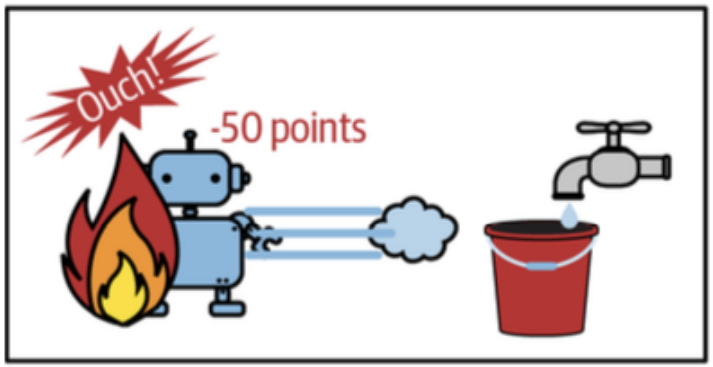
Reinforcement Learning Basics

- The learning system is called an agent; it interacts with an environment by taking actions.
- After each action, the agent receives a reward (or penalty) and observes the new state.
- The agent's goal is to learn a policy: a strategy mapping states to actions that maximizes cumulative reward over time.

Reinforcement learning loop



1. Observe
2. Select action using policy



3. Action!
4. Get reward or penalty

5. Update policy (learning step)
6. Iterate until an optimal policy is found

RL Examples

- Robots learn to walk by trial and error, adjusting their policy based on balance and forward progress.
- AlphaGo learned a strong Go policy by analyzing human games and playing many games against itself.
- During deployment (e.g., matches against human champions), learning is typically turned off; the agent exploits its learned policy.

Batch Versus Online Learning

Batch Learning

- In batch learning, models are trained on all available data at once, typically offline.
- After training, the model is deployed and used without further learning; this is offline learning.
- Performance degrades over time if the world changes; periodic retraining on updated data is necessary.
- Retraining from scratch on full data can be expensive in time and resources, and may not be feasible for very large datasets.

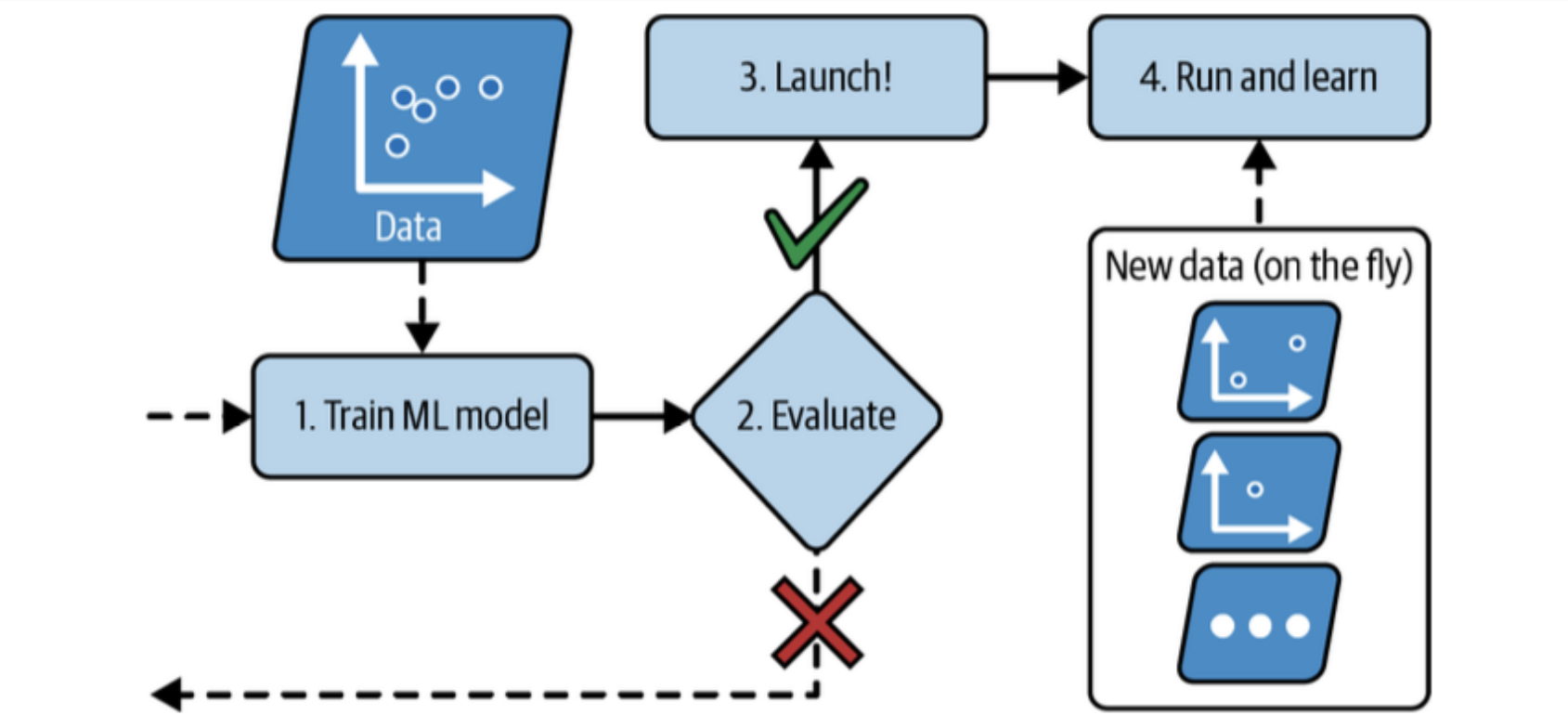
Model Rot and Data Drift

- Even for stable tasks, data distributions shift slowly (e.g., image formats, camera qualities, popular breeds).
- For fast-changing domains (e.g., financial markets), model performance can decay quickly if not updated.
- Batch retraining schedules (daily, weekly, etc.) must balance recency, computational cost, and operational constraints.

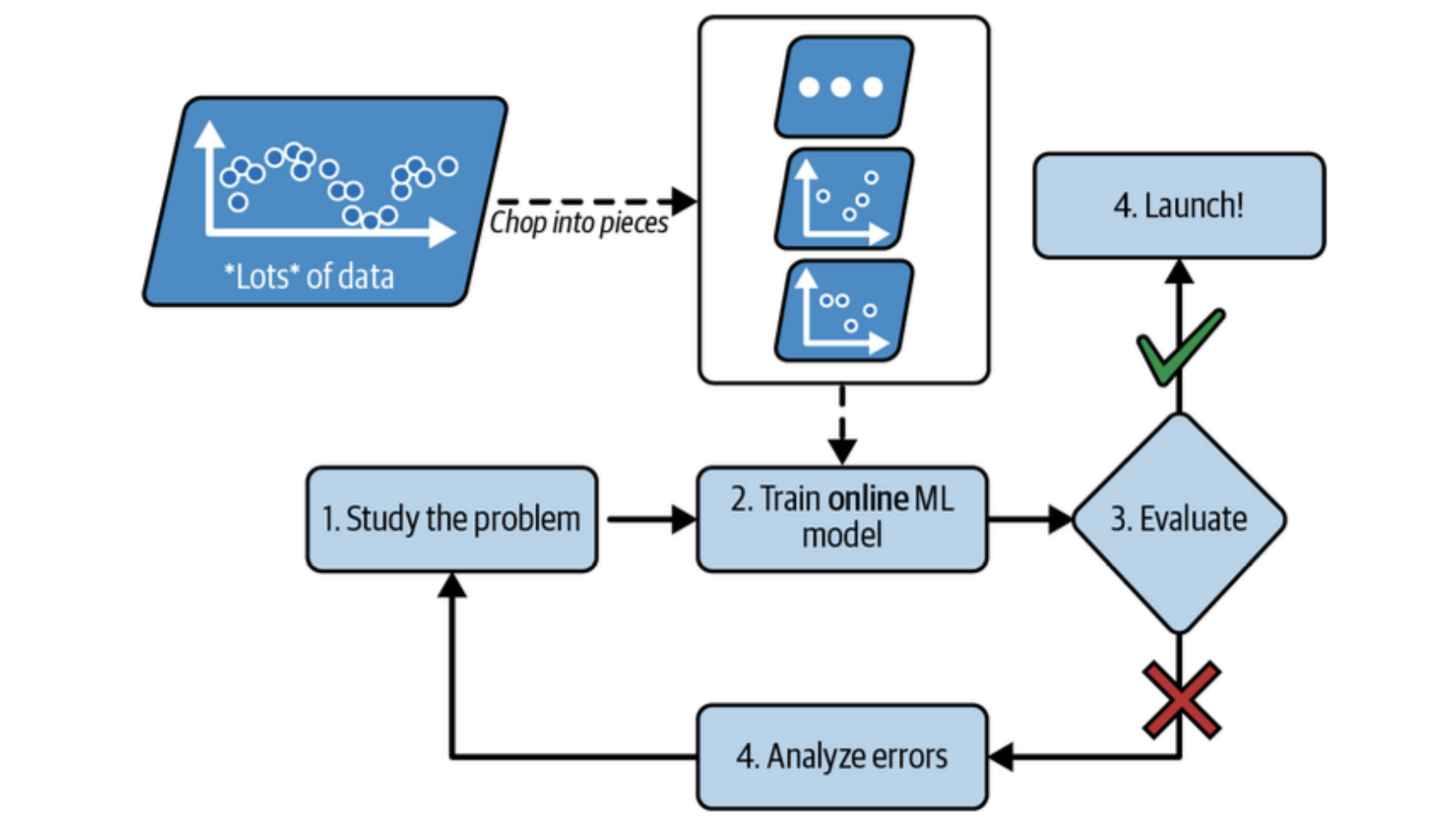
Online Learning

- Online learning updates the model incrementally as each new instance (or mini-batch) arrives.
- Each update is fast and cheap, enabling continuous adaptation to new data.
- Online methods can handle massive datasets that cannot fit into memory (out-of-core learning).
- They are suitable for constrained environments (e.g., mobile devices, remote robots) where storage and compute are limited.

Online learning



Out-of-core learning



Learning Rate and Data Quality

- Learning rate controls how fast an online model adapts to new data: high rate adapts quickly but forgets old patterns.
- Low learning rate makes the model more stable but slower to respond to real changes.
- Bad data (e.g., sensor failures, adversarial input) can quickly degrade performance, especially with high learning rates.
- Monitoring performance and input data, with the ability to pause learning and roll back, is crucial.

Instance-Based vs Model-Based Learning

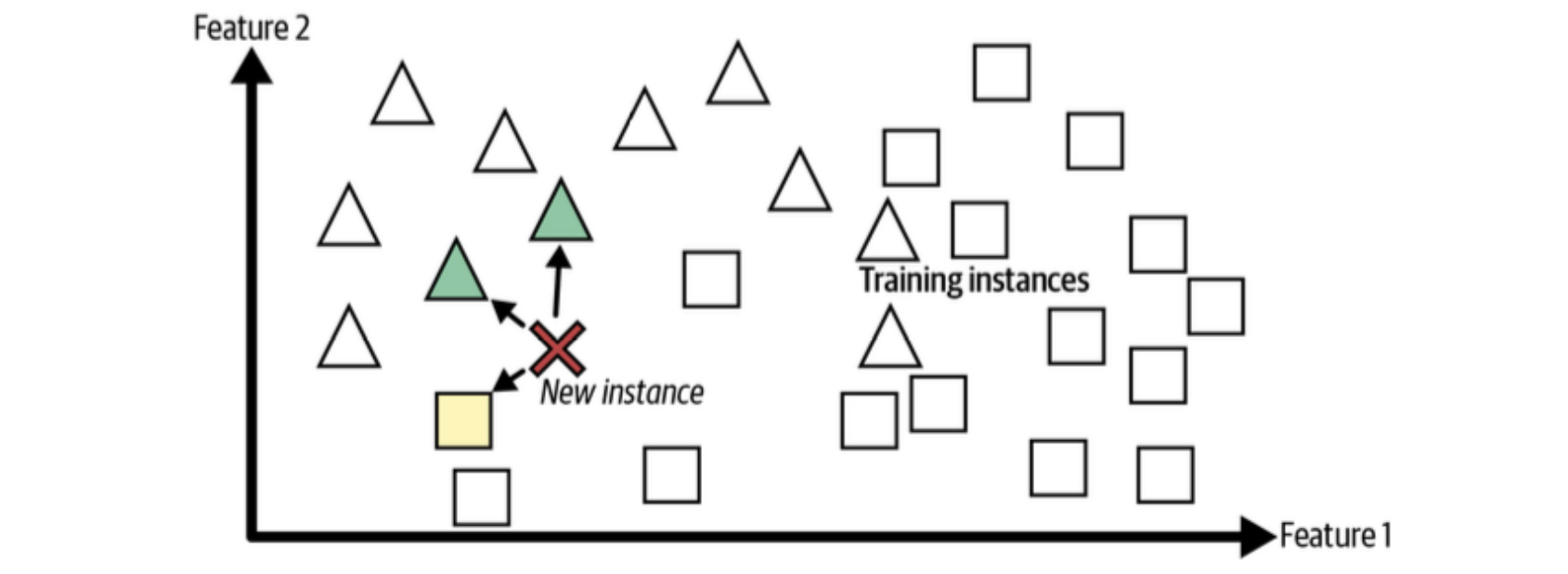
Generalization: The Core Goal

- Most ML tasks aim to generalize: perform well on new, unseen examples.
- Good performance on training data is necessary but not sufficient; we care about out-of-sample performance.
- Two main ways to generalize: instance-based learning and model-based learning.

Instance-Based Learning

- The system "memorizes" training instances and compares new cases to them using a similarity measure.
- Predictions are made based on the most similar stored examples (e.g., majority class among nearest neighbors).
- Example: k-nearest neighbors classifier or regressor.
- Similarity can be defined in various ways (e.g., number of shared words, Euclidean distance in feature space).

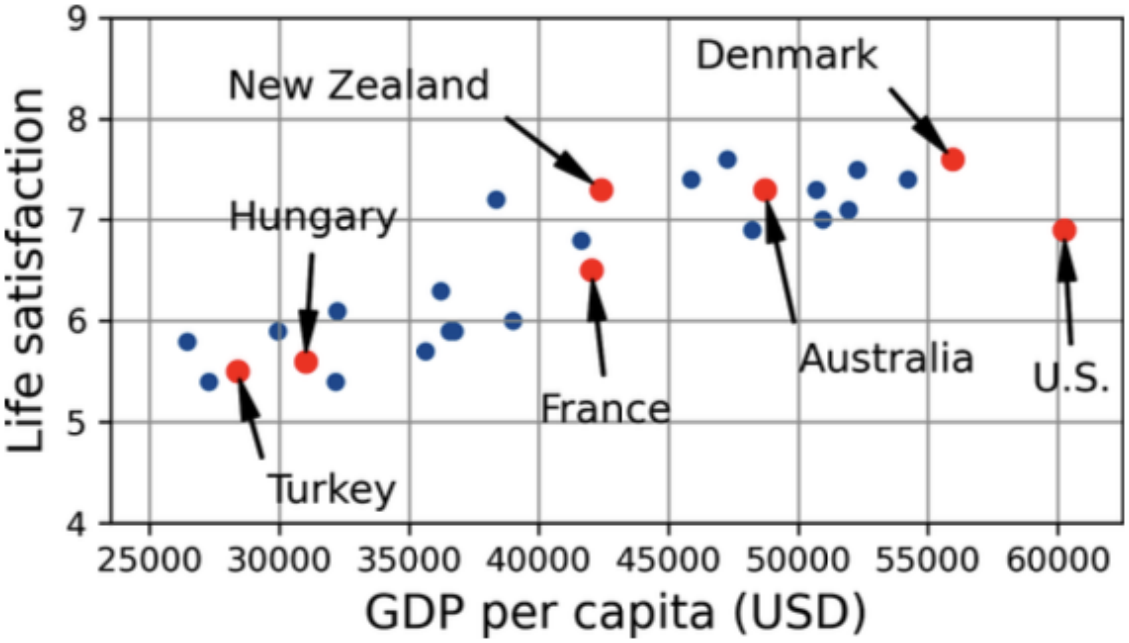
Instance-based learning



Model-Based Learning: High-Level Idea

- Model-based learning builds an explicit model that captures patterns in the data, then uses it to make predictions.
- Example: learn a linear relationship between GDP per capita and life satisfaction using regression.
- The workflow: select a model family, choose a performance measure, find the parameters that best fit the data, and use the model for inference.

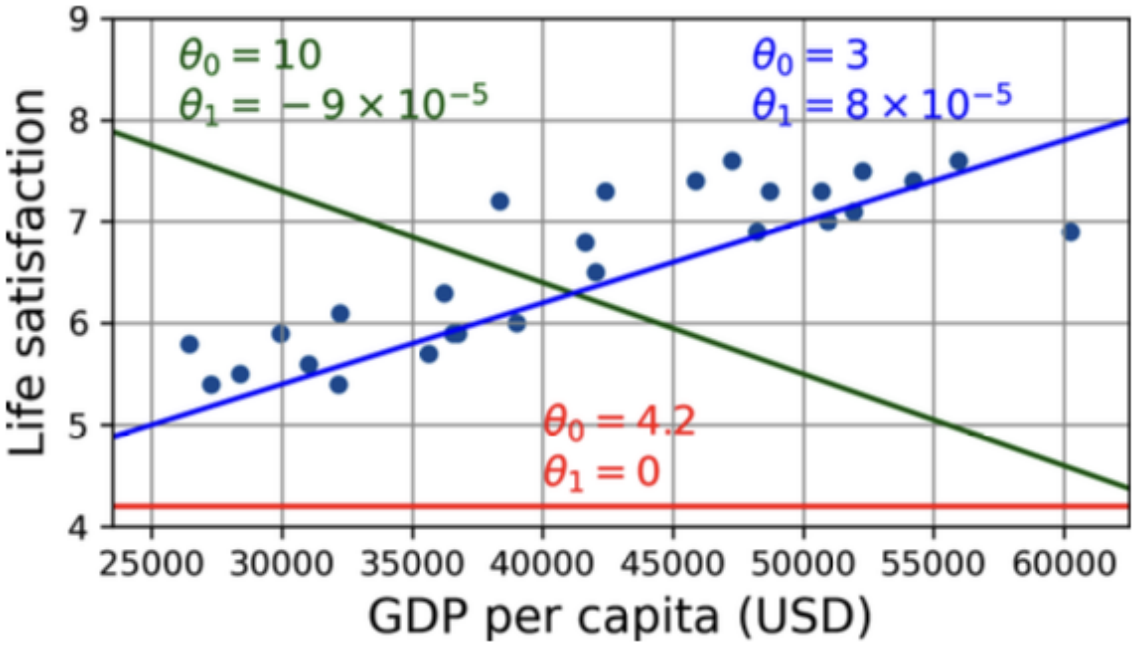
GDP per capita vs life satisfaction



Linear Model Example

- Choose a simple linear model: life_satisfaction $= \theta_0 + \theta_1 \cdot \text{GDP_per_capita}$.
- θ_0 (intercept) and θ_1 (slope) are model parameters to be learned from data.
- Different choices of θ_0 and θ_1 yield different lines; we want the line that best fits the training data.

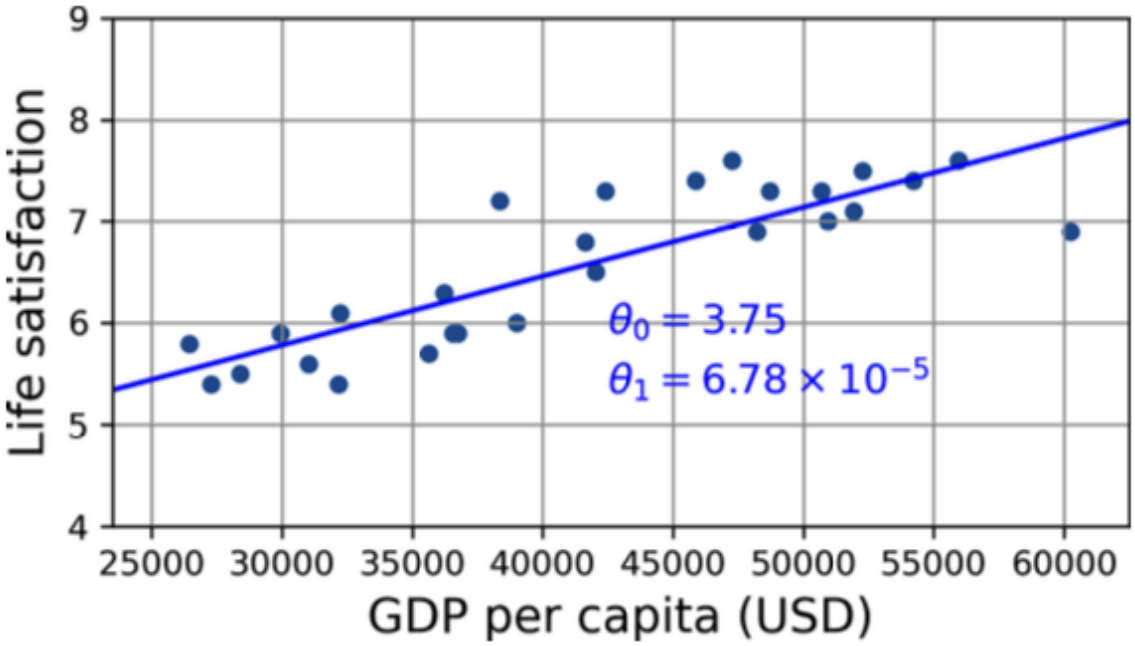
Candidate linear models



Choosing Parameters and Training

- Define a performance measure: often a cost function measuring distance between predictions and actual values.
- Linear regression algorithms search for θ_0 and θ_1 that minimize this cost on the training data.
- Training the model means running the algorithm to find parameter values that best fit the data according to the chosen measure.

Best fit


Life satisfaction
GDP per capita (USD)
$\theta_0 = 3.75$
$\theta_1 = 6.78 \times 10^{-5}$
9
8
7
6
5
4
25000
30000
35000
40000
45000
50000
55000
60000

From Training to Inference

- Once trained, the model can predict life satisfaction for new countries given their GDP per capita.
- Example: plug Cyprus's GDP per capita into the learned line to estimate its average life satisfaction.
- Instance-based alternative: find countries with most similar GDP and average their life satisfaction values (k-nearest neighbors regression).

Summary of a Typical ML Project

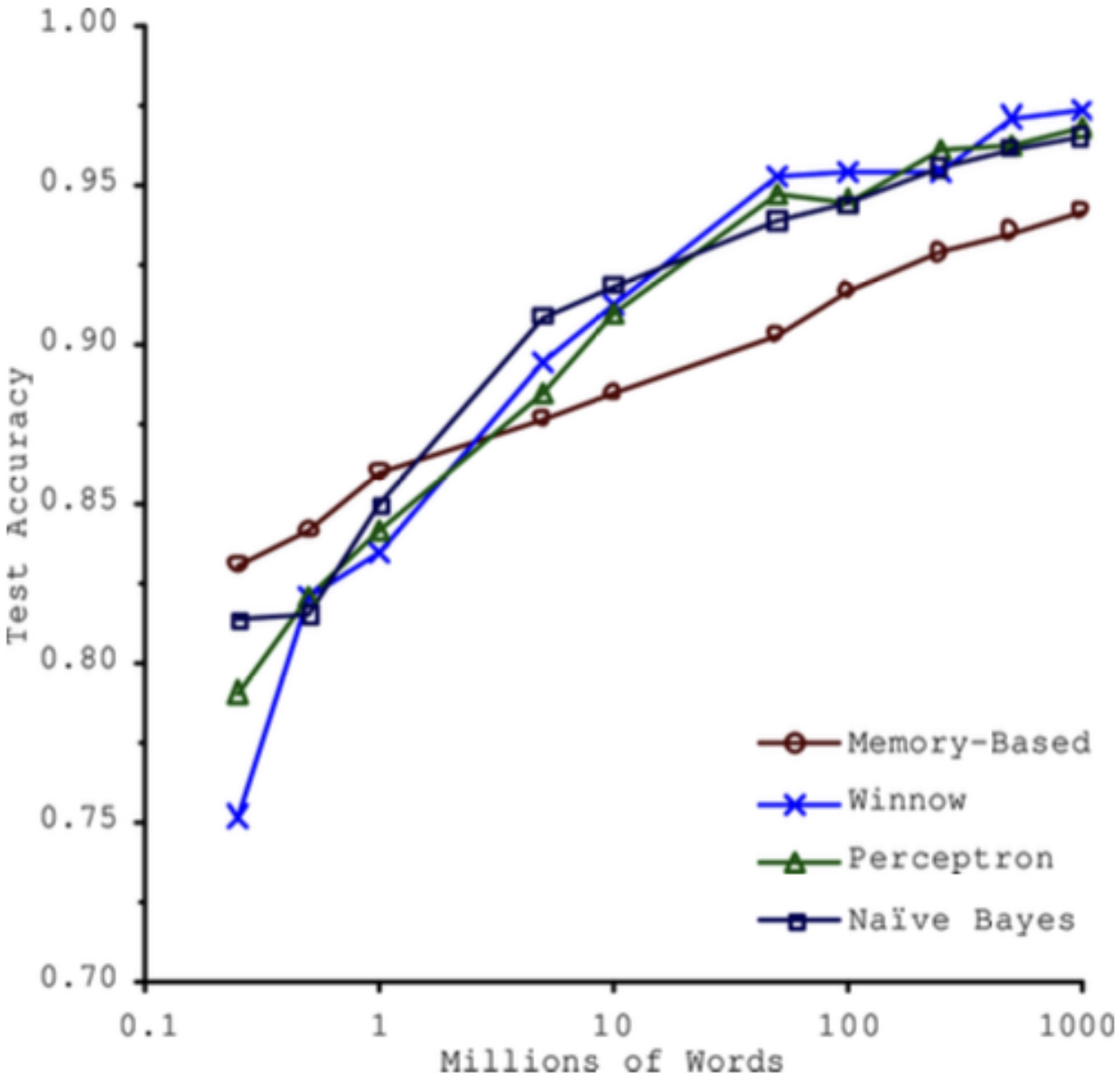
- Study the data and understand its structure and quality.
- Select a model family appropriate for the task and data.
- Train the model by finding parameter values that minimize a cost function on the training set.
- Use the trained model to make predictions (inference) on new cases, aiming for good generalization.

Main Challenges of Machine Learning

Data Quantity and the Power of Data

- Many ML algorithms need large datasets to perform well, especially for complex tasks like image and speech recognition.
- Studies show that with enough data, very different algorithms can converge to similar performance.
- This suggests a trade-off between investing in better algorithms and investing in more and better data.

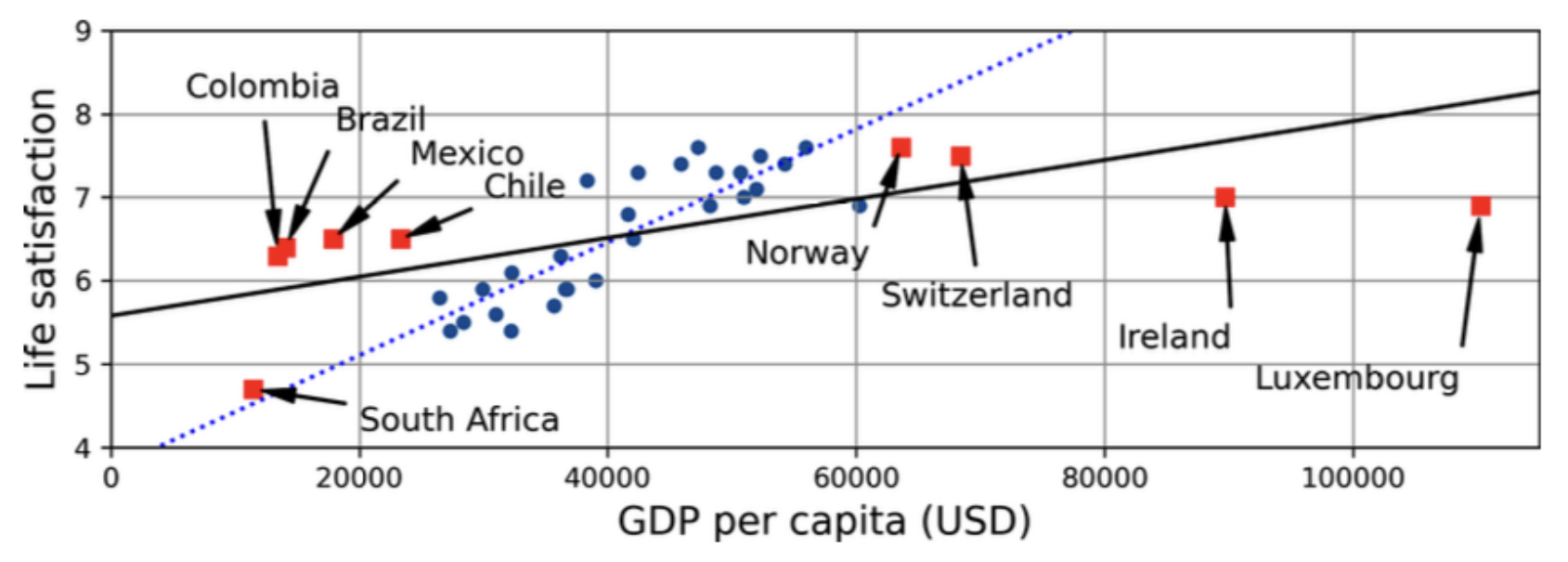
Similar performances



Nonrepresentative Training Data

- To generalize well, training data must be representative of the cases the model will encounter in production.
- If training data lacks very poor or very rich countries, a GDP–life satisfaction model may perform poorly at extremes.
- Sampling noise (from small samples) and sampling bias (from flawed collection methods) both hurt representativeness.

Limitations of linear models



Sampling Bias Examples

- Famous 1936 US election poll: biased toward wealthier individuals due to sampling from telephone directories and magazines.
- Nonresponse bias: people choosing not to respond can systematically differ from those who do.
- Web search-based datasets may be biased toward popular content or specific regions.

Poor-Quality Data

- Errors, outliers, and noise make it harder for the model to discover true patterns.
- Data cleaning (fixing or removing outliers, handling missing values) is often a large part of a data scientist's work.
- Options for missing values include dropping instances, dropping features, or imputing values (e.g., median).

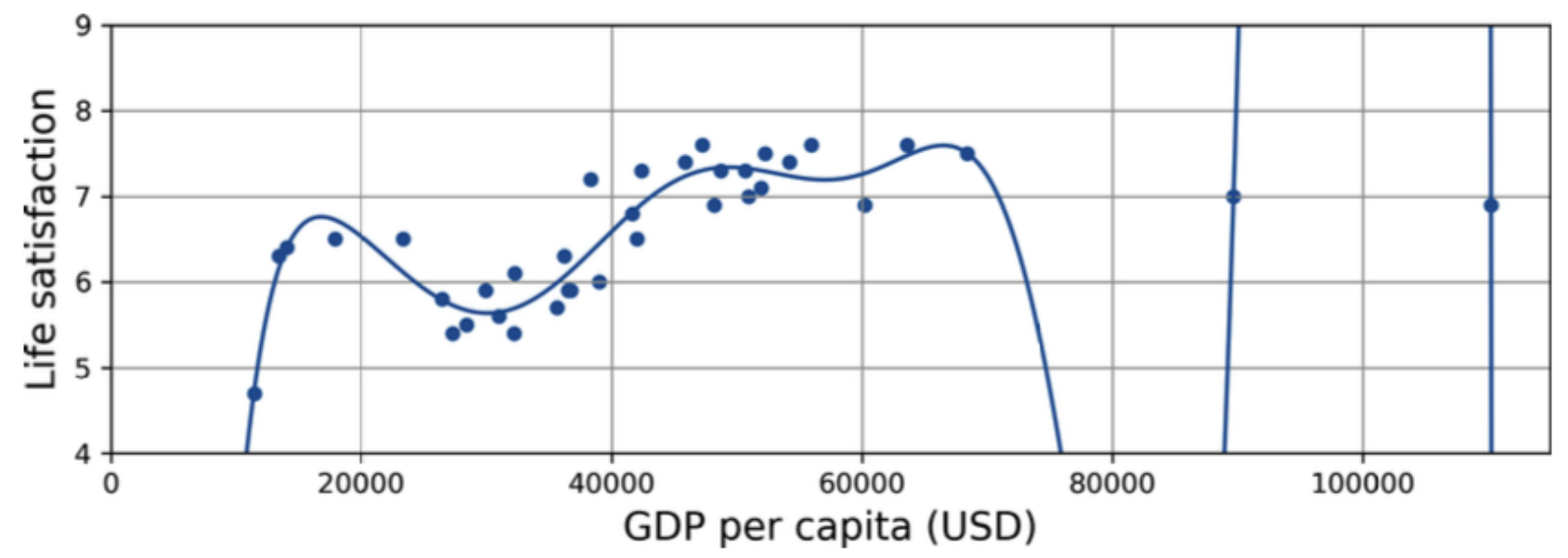
Irrelevant and Missing Features

- Without enough relevant features and with too many irrelevant ones, models will struggle to learn useful patterns.
- Feature engineering:
- Feature selection: choosing the most informative existing features.
- Feature extraction: combining features or using dimensionality reduction to create better features.
- Creating new features by collecting additional data can dramatically improve performance.

Overfitting the Training Data

- Overfitting occurs when a model fits the training data too closely, capturing noise or chance patterns instead of true structure.
- Overfitted models have very low training error but high error on new data.
- Complex models (e.g., high-degree polynomials, deep networks) are more prone to overfitting, especially with small or noisy datasets.

Overfitting



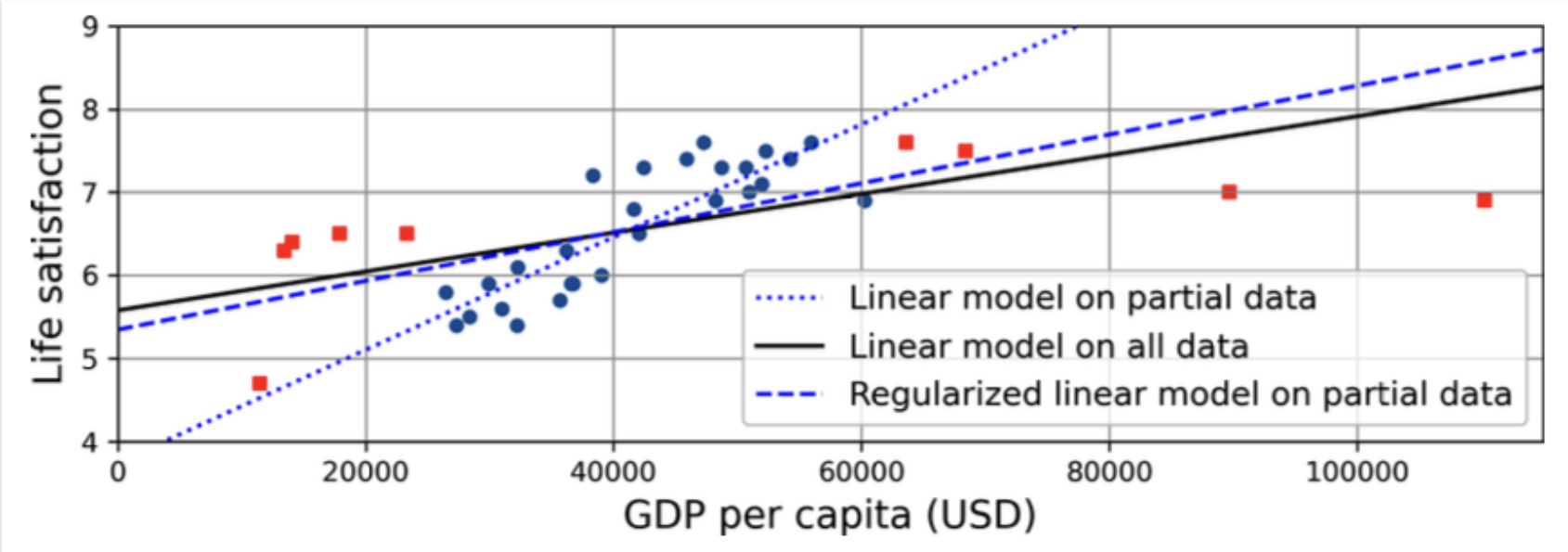
Causes and Remedies for Overfitting

- Causes: overly complex models relative to dataset size and noise; many irrelevant features.
- Remedies:
- Simplify the model (fewer parameters, simpler architecture, fewer features).
- Gather more training data if possible.
- Reduce noise in the training data (better collection, cleaning).
- Apply regularization to constrain the model.

Regularization and Hyperparameters

- Regularization adds constraints or penalties to keep model parameters small or simple, reducing overfitting risk.
- The strength of regularization is controlled by a hyperparameter, set before training and kept fixed during training.
- Strong regularization yields very simple models; weak regularization allows more complex fits.
- Choosing the right regularization level balances training fit and generalization.

Regularized model



Underfitting the Training Data

- Underfitting happens when the model is too simple to capture the underlying structure of the data.
- Underfit models perform poorly even on training data, as well as on new data.
- Remedies:
- Use a more powerful model with more parameters or more expressive architecture.
- Provide better features via feature engineering.
- Reduce regularization strength if it is too high.

Testing and Validating

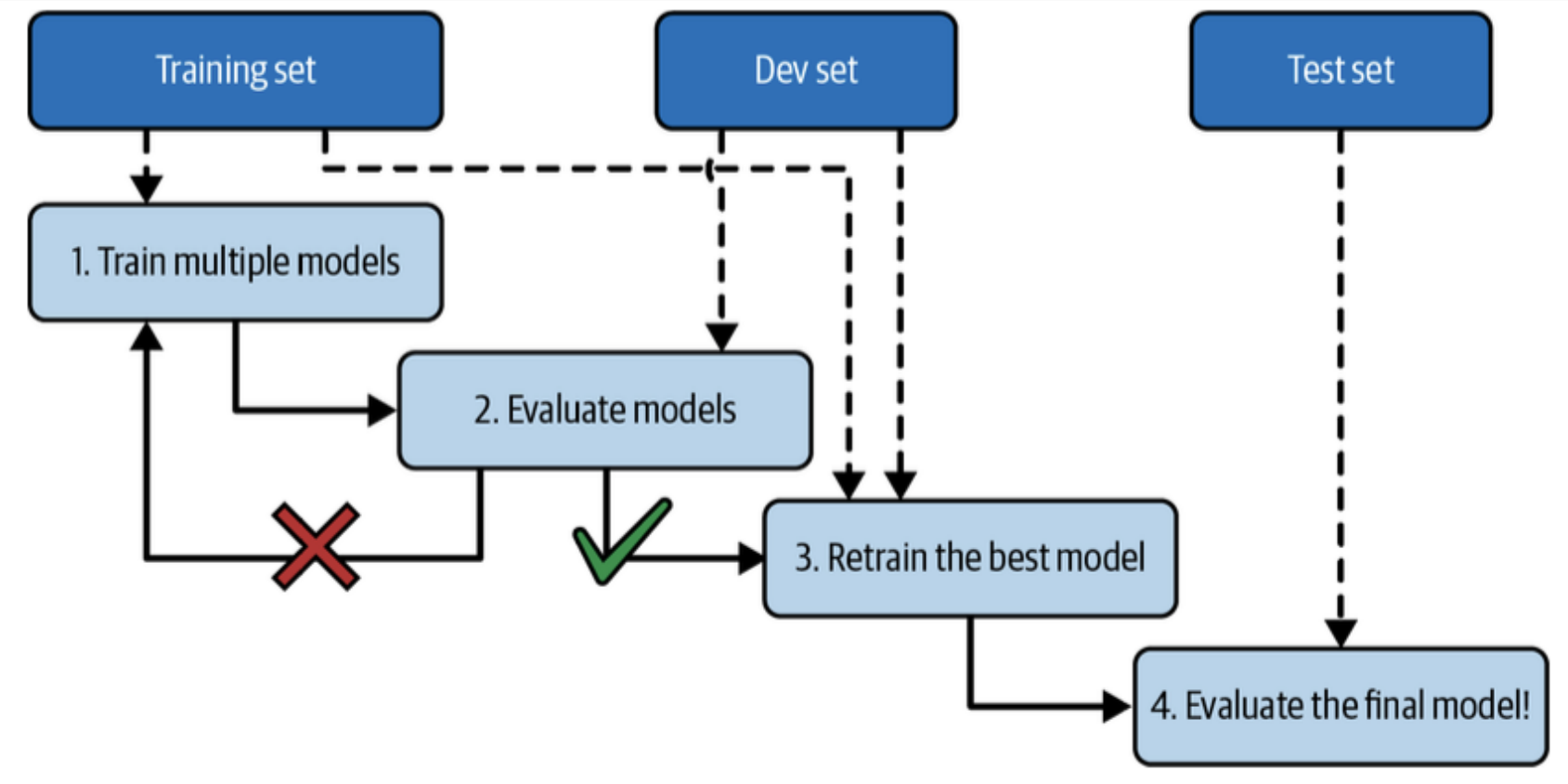
Generalization Error and Test Sets

- Generalization error (out-of-sample error) is the error a model makes on new data drawn from the same distribution as the training data.
- To estimate it, split data into a training set and a test set.
- Train the model on the training set, then evaluate it on the test set, which the model has never seen.
- Large differences between training and test errors typically indicate overfitting.

Holdout Validation and Model Selection

- When comparing models or tuning hyperparameters, using the test set repeatedly can lead to overly optimistic estimates.
- Holdout validation introduces a third split: training set, validation (dev) set, and test set.
- Models and hyperparameters are chosen based on validation performance, then retrained on full training data and evaluated once on the test set.

Holdout validation



Cross-Validation

- If the validation set is small, performance estimates are noisy; if it is large, less data remains for training.
- Cross-validation splits the data into multiple folds, training and validating models on each fold in turn.
- Averaging performance across folds yields more reliable estimates and uses data more efficiently.
- The trade-off is increased computation time proportional to the number of folds.

Data Mismatch and Train-Dev Sets

- Sometimes training data is easy to obtain but differs from real-world production data (e.g., web images vs mobile photos).
- Validation and test sets should always be representative of production data, even if small.
- A train-dev set, drawn from the training distribution, can help distinguish overfitting from distribution mismatch issues.

Train
Train-dev
Dev
Test

No Free Lunch Theorem

- The No Free Lunch theorem states that, without assumptions about the data, no model is universally best across all problems.
- For some datasets, linear models perform best; for others, neural networks or other models are better.
- In practice, we make reasonable assumptions about the data and evaluate a small set of suitable model families.

Stepping Back – Big Picture

- ML is about improving a task's performance by learning from data instead of hand-coding rules.
- Systems vary along axes: supervised vs unsupervised vs self-/semi-supervised vs reinforcement; batch vs online; instance-based vs model-based.
- Typical workflow: collect and clean data, choose features and model, train and regularize, evaluate and validate, and then iterate.
- Success depends on representative, high-quality data, well-chosen features, appropriately complex models, and careful evaluation.